

Angela Luca, 1100008
Benslimane Bilal, 1094638



UNIVERSITÀ
DI TORINO

Turno 2

Basi di dati - Progetto A.A 2024/2025

Parchi naturali



Indice

1. Introduzione.....	3
1.1 Descrizione del progetto.....	3
1.2 Glossario dei termini.....	5
1.3 Requisiti strutturali (Business Rules).....	6
1.3.1 Relazioni Utenti e Persone.....	6
1.3.2 Relazioni Parchi e Gestione.....	6
1.3.3 Relazioni Tour e Tragitto.....	7
1.3.4 Relazioni Guide.....	7
2. Progettazione logica.....	8
2.1 Tavola dei volumi.....	8
2.2 Tavola delle operazioni.....	9
3. Schema ER.....	10
3.1 Schema ER Completo con Business Rules.....	10
3.2 Ristrutturato.....	11
4. Schema Relazionale.....	12
4.1 Entità.....	12
4.2 Relazioni.....	12
5. Ristrutturazione schema E-R.....	13
5.1 Analisi delle ridondanze.....	13
5.2 Eliminazione delle generalizzazioni.....	14
5.3 Partizionamento/Accorpamento.....	14
5.4 Eliminazione degli attributi composti.....	14
5.5 Scelta degli identificatori principali.....	14
6. Codice SQL.....	15
6.1 Creazione DB e tuple.....	15
6.2 Inserimento in tabella.....	27
6.3 Query Test.....	32
6.4 Test dei vincoli in SQL.....	35

1.Introduzione

1.1 Descrizione del progetto



Il progetto ha l'obiettivo di realizzare un sistema informativo dedicato alla gestione e all'organizzazione delle **aree protette italiane**.

Il sistema raccoglie e struttura tutte le informazioni relative ai **parchi naturali, riserve e aree marine protette**, consentendo una consultazione ordinata e aggiornata.

Per chi desidera visitare l'area, deve fornire informazioni sui centri visita sufficienti per facilitare l'organizzazione di visite e percorsi (esempio a sinistra).

Per ogni parco vengono memorizzati dati come:

- nome, regione, tipologia (parco nazionale, riserva, ecc.);
- ente gestore e provvedimento di istituzione;
- superficie terrestre e marina, chilometri di costa e altre informazioni ambientali;
- adesione a programmi di certificazione come la **Carta Europea del Turismo Sostenibile (CETS)**.

Ogni area protetta dispone di una sezione dedicata con **notizie aggiornate**, complete di date, testo e immagini, oltre a informazioni pratiche sui **centri visita** utili per l'organizzazione di escursioni e attività.

Il sistema fornisce inoltre un elenco di **strutture ricettive** nei pressi dei parchi, con dettagli su:

- indirizzo e contatti;
- servizi e trattamenti disponibili;
- eventuale impegno ecologico e adesione alla CETS.

Le strutture possono ricevere **richieste di prenotazione** da parte degli utenti registrati, che specificano periodo e numero di ospiti.

Gli **itinerari escursionistici** sono descritti nel dettaglio, indicando segnaletica, punto di partenza, durata, livello di difficoltà e modalità di percorrenza (a piedi, in bici, a cavallo).

Ogni itinerario può essere associato a una o più **guide abilitate**, di cui vengono gestite licenze, valutazioni e disponibilità tramite calendario.

Il sistema permette di:

- gestire **tour guidati** con orari, partecipanti e guide assegnate;
- raccogliere **feedback e valutazioni** sui percorsi e sulle guide;
- registrare le **presenze dei visitatori** nei parchi, con orari, itinerari percorsi e tipologia di visitatore (singolo, gruppo scolastico, senior, bambini, ecc.);
- fornire ai gestori strumenti di analisi basati sui dati storici delle visite.

In questo modo, il progetto mira a creare una piattaforma completa e dinamica per la gestione integrata delle aree protette, favorendo al tempo stesso **la tutela ambientale, il turismo sostenibile e la valorizzazione del patrimonio naturale italiano**.

1.2 Glossario dei termini

Termine	Descrizione	Collegamenti
Parco	Area protetta costituita da vari tipi di biomi in base al luogo.	Centro Visita, Ente, News, Visita
Centro Visita	Centro affiliato al parco in cui è attiva la prenotazione della giornata all'area protetta.	Parco, Persona
Ente	Azienda che si occupa di gestire l'area protetta in base agli standard europei. Può essere classificato in base al CETS	Parco
News	Informazioni che si vengono a creare in base ai recenti avvenimenti all'interno del parco.	Parco
Persona	Individuo (bambino o senior, nel primo caso in gruppo) che si prenota al parco registrandosi in un apposito sito	Visita, Centro Visita, Utente, Gruppo, Riservare
Gruppo	Insieme di individui che appartengono ad una classe scolastica o ad un gruppo	Persona
Riservare	Prenotazione in una Struttura Ricettiva	Persona, Struttura Ricettiva
Struttura Ricettiva	Luogo in cui le persone possono pernottare per una serie di giorni. Le strutture sono esterne dal parco e possono avere anche loro la certificazione CETS	Riservare
Utente	La persona, o il referente del gruppo, che si registra nel sito del parco.	Prenota, Feedback, Persona
Tour	Insieme di percorsi del parco organizzati dalle guide	Prenota, Guida, Tragitti
Feedback	Recensione da parte di un Utente della sua esperienza vissuta nel Parco	Guida, Tragitti, Utente
Calendario	Appartiene alle Guide in cui viene segnata la loro disponibilità e vengono gestiti i vari turni.	Guida

Guida	Persona che lavora all'interno del Parco e gestisce le varie escursioni al suo interno.	Calendario, Tour, Tragitti, Feedback
Tragitto	Percorso disponibile da fare nel Parco, a seconda delle modalità di spostamento (a bici, a piedi)	Guida, Feedback, Tour
Visita	Persona che visita il Parco	Parco, Persona
Prenota	Persona che prenota la giornata o le giornate al Parco	Tour, Utente

1.3 Requisiti strutturali (Business Rules)

1.3.1 Relazioni Utenti e Persone

- Un UTENTE è collegato ad una PERSONA, e una PERSONA può essere collegata ad un UTENTE.
- Un GRUPPO deve essere "affidato a" esattamente una PERSONA . Una PERSONA può essere referente di zero o più GRUPPO.
- Una PERSONA può "soggiornare" in zero o una STRUTTURA_RICETTIVA, e una struttura può ospitare (soggiorna) zero o più PERSONA.
- Una PERSONA può "visitare" un PARCO, e un PARCO può essere visitato da zero e più PERSONA.
- Una PERSONA "si rivolge a" almeno un CENTRO_VISITA, e un CENTRO_VISITA accoglie una o più PERSONA.

1.3.2 Relazioni Parchi e Gestione

- Un PARCO deve "appartenere" esattamente a una Categoria (es. NATURALE, RISERVA, ecc.).
- Un PARCO deve essere in "gestione" da almeno un ENTE. Un ENTE può gestire uno o più PARCO.
- Una NEWS deve "riguardare" esattamente un PARCO. Un PARCO può essere oggetto almeno una NEWS.

- Un PARCO possiede uno o più CENTRO_VISITA, e un CENTRO_VISITA lavora su uno o più parchi

1.3.3 Relazioni Tour e Tragitto

- Un UTENTE può "prenotare" zero o più TOUR, e un TOUR può essere prenotato da più UTENTE.
- Un FEEDBACK deve essere "scritto" da esattamente un UTENTE. Un UTENTE può scrivere zero o più FEEDBACK.
- Un FEEDBACK deve "riguardare" esattamente un TRAGITTO. Un TRAGITTO può avere zero o più FEEDBACK.
- Un TOUR può "prevedere" uno o più TRAGITTO, e un TRAGITTO può essere previsto da zero o più TOUR.

1.3.4 Relazioni Guide

- Un TRAGITTO può avere zero o più GUIDA "assegnate", e una GUIDA può essere assegnata a zero o più TRAGITTO.
- Una GUIDA deve avere almeno una (o più) "disponibilità" nel CALENDARIO. Un periodo di CALENDARIO si riferisce a almeno una GUIDA.

2. Progettazione logica

2.1 Tavola dei volumi

Concetto	Tipo	Volume Stimato	Motivazione
Parco	Entità	Basso (es. 10 - 100)	Il numero di parchi gestiti dal sistema è limitato e stabile (anagrafica statica).
Centro_visita	Entità	Basso (es. 50 - 500)	Strettamente legato ai parchi (un parco può avere più centri). Volume stabile.
Ente	Entità	Molto Basso (es. < 50)	Il numero di enti gestori (regioni, ministeri, ecc.) è molto limitato.
News	Entità	Medio (es. 1.000 - 10.000)	Le news si accumulano nel tempo, ma non sono l'elemento transazionale principale.
Persona	Entità	Alto (es. 100.000+)	Anagrafica principale che include tutti i visitatori (anche quelli non registrati come utenti, es. membri di gruppi) e lo staff.
Gruppo	Entità	Medio (es. 10.000 - 50.000)	Registra le prenotazioni di gruppi (es. scolaresche). Meno numeroso delle singole persone.
Struttura_ricettiva	Entità	Medio (es. 1.000 - 5.000)	Anagrafica degli hotel/B&B convenzionati. Numero definito e relativamente stabile.
Utente	Entità	Alto (es. 50.000+)	Rappresenta gli utenti con login. È un sottoinsieme di Persona ma comunque ad alto volume.
Tour	Entità	Alto (es. 100.000+)	Ogni record è un'istanza di un tour (con data e ora). Si accumulano rapidamente nel tempo.
Feedback	Entità	Alto (es. 100.000+)	Volume proporzionale al numero di utenti e di attività svolte (tragitto).
Calendario	Entità	Medio-Alto (es. 50.000+)	Ogni guida ha molti slot di disponibilità/impegno. Cresce con le guide e con il tempo.
Guida	Entità	Basso (es. 100 - 1.000)	Il numero di guide professioniste abilitate è un'anagrafica limitata e controllata.

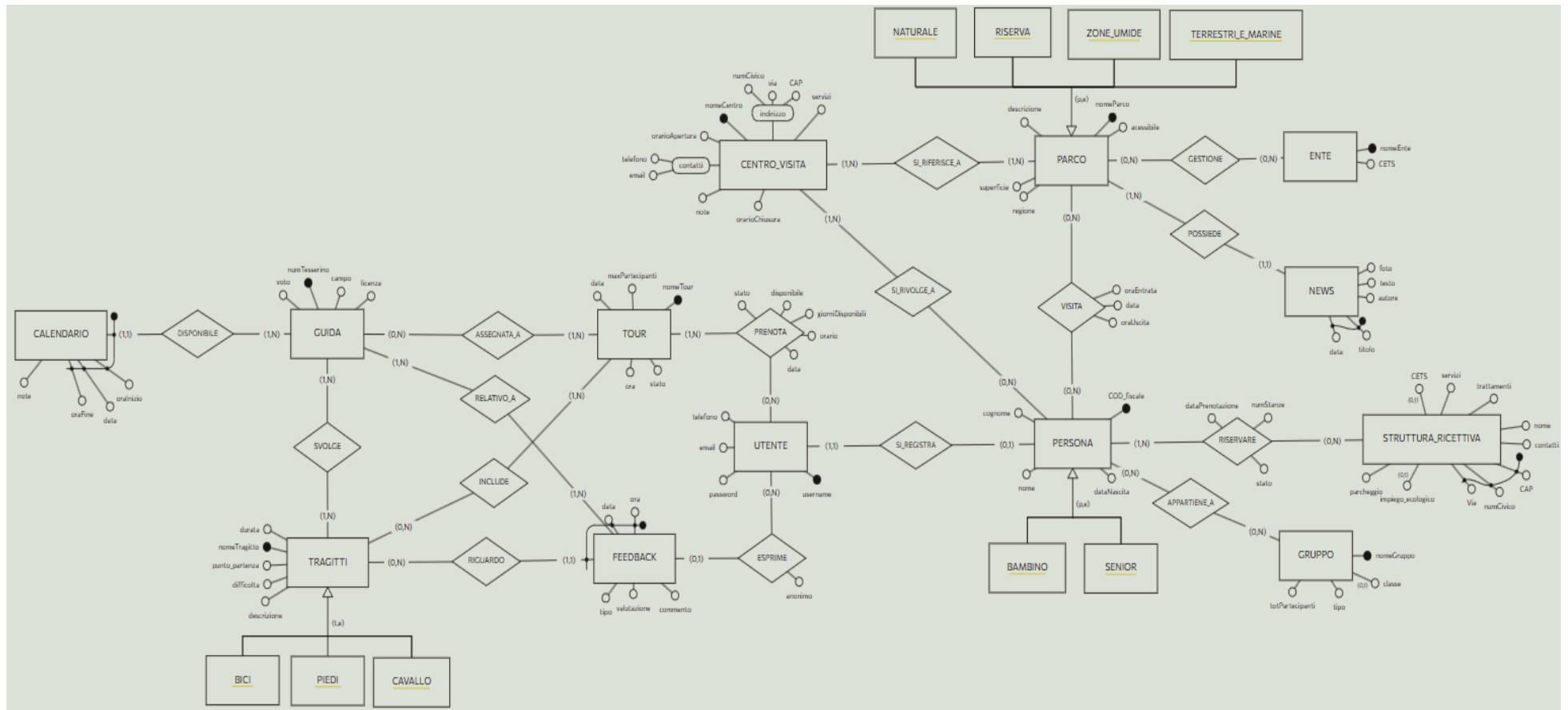
Tragitto	Entità	Medio (es. 500 - 5.000)	Anagrafica dei sentieri/percorsi. Stabile, non cresce quotidianamente.
Visita	Relazione	Molto Alto (es. 1.000.000+)	Tabella transazionale che traccia ogni ingresso di ogni persona. È una delle tabelle a crescita più rapida.
Esprime	Relazione	Alto (es. 100.000+)	Assumendo che ogni feedback sia espresso da un utente, il volume è identico a quello dell'entità Feedback.
Prenota	Relazione	Molto Alto (es. 1.000.000+)	Tabella transazionale che registra ogni prenotazione utente-tour. Cresce molto velocemente.
Riservare	Relazione	Alto (es. 500.000+)	Registra le prenotazioni delle strutture. È un volume transazionale elevato, legato ai pernottamenti.

2.2 Tavola delle operazioni

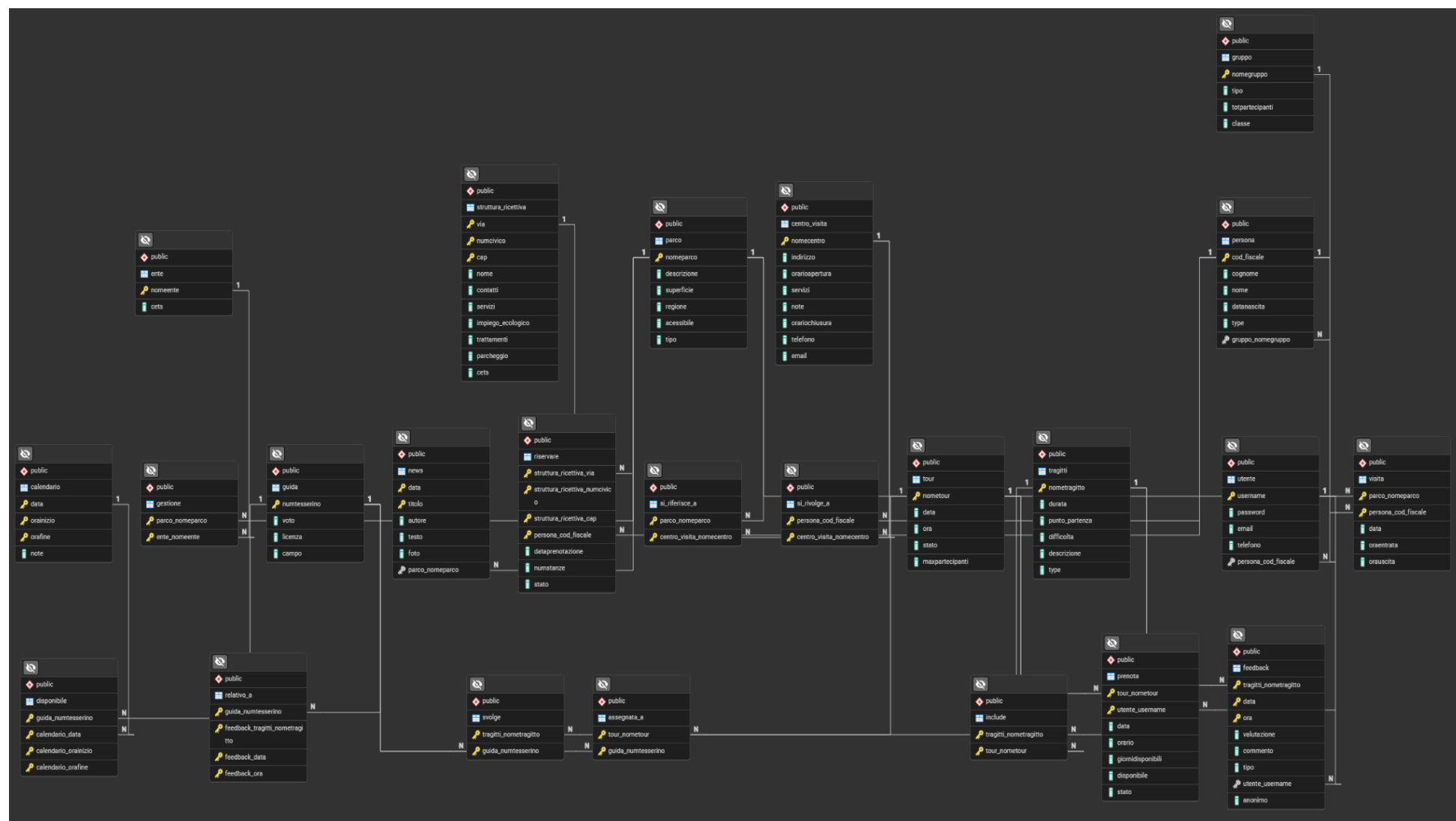
#	Descrizione	Tipo	Frequenza (al gg)
1	Registrazione di un nuovo utente	Interattiva (Scrittura)	50
2	Ricerca di un Parco e visualizzazione dettagli	Interattiva (Lettura)	2000
3	Inserimento di una nuova Visita (ingresso al parco)	Interattiva (Scrittura)	5000
4	Prenotazione di un Tour guidato	Interattiva (Scrittura)	500
5	Calcolo del totale dei partecipanti di un Gruppo	Batch (Lettura)	100
6	Inserimento Feedback su un Tragitto	Interattiva (Scrittura)	300
7	Ricerca Guide disponibili per data	Interattiva (Lettura)	200

3. Schema ER

3.1 Schema ER Completo con Business Rules



3.2 Ristrutturato



4. Schema Relazionale

4.1 Entità

Parco(**nomeParco(pk)**, descrizione, superficie, regione, accessibile, tipo)

Centro_visita(**nomeCentro(pk)**, servizi, note, orarioApertura, orarioChiusura, telefono, email, indirizzo)

Ente(**nomeEnte(pk)**, CETS)

News(**data(pk)**, **titolo(pk)**, foto, testo, autore)

Persona(**COD_fiscale(pk)**, nome, cognome, dataNascita, tipo, *nomeGruppo*)

Gruppo(**numGruppo(pk)**, tipo, classe)

Struttura_ricettiva(**numCivico(pk)**, **via(pk)**, **CAP(pk)**, contatti, servizi, trattamenti, CETS*, parcheggio, impiego_ecologico)

Utente(**username(pk)**, password, email, telefono, *CODfiscale*)

Tour(**nomeTour(pk)**, maxPartecipanti, data, ora, stato)

Feedback(**idFeedback(pk)**, tipo, valutazione, commento, data, ora, autore, Feedback_Tragitto_FK)

Calendario(**idCalendario(pk)**, data, oraInizio, oraFine, note)

Guida(**numTesserino(pk)**, nome, campo, licenza)

Tragitto(**idTragitto(pk)**, durata, nomeTragitto, punto_partenza, difficoltà, descrizione, tipo)

4.2 Relazioni

Visita(**nomeParco(pk)**, **COD_fiscale(pk)**, data, oraInizio, oraFine)

Esprime(**username(pk)**, **idFeedback(pk)**, anonimo)

Prenota(**username(pk)**, **nomeTour(pk)**, stato, dataPrenotazione, giorniDisponibili, orario)

Riservare(**COD_fiscale(pk)**, **nome(pk)**, **dataPrenotazione(pk)**, numPersone, stato)

5. Ristrutturazione schema E-R

Nella fase di affinamento dello schema ER, in particolare durante la ristrutturazione delle generalizzazioni, sono state apportate modifiche significative:

- **Gerarchie:** Le gerarchie relative alle entità PARCO, TOUR e PERSONA sono state accorpate nelle rispettive entità genitore. La distinzione tra le sottoclassi è ora gestita tramite un attributo discriminante (tipo), soluzione che semplifica la struttura ed evita la gestione di vincoli complessi sui valori.
- **Attributi Composti:** Per l'attributo Indirizzo, si è scelto di concatenare i sotto-attributi in un unico campo stringa. Al contrario, i sotto-attributi di Contatti sono stati mantenuti separati per garantire una maggiore correttezza e granularità dell'informazione.
- **Relazioni:** La relazione ESPRIME è stata semplificata trasferendo l'attributo anonimo direttamente all'interno dell'entità FEEDBACK.

5.1 Analisi delle ridondanze

È stata individuata una ridondanza relativa all'attributo **totPartecipanti** nell'entità GRUPPO. Tale dato è *derivabile* contando le occorrenze della relazione tra PERSONA e GRUPPO.

- **Attributo:** **totPartecipanti** (in Gruppo)
- **Operazioni coinvolte:**
 - *Lettura:* Visualizzazione dettagli Gruppo (Op. frequente).
 - *Scrittura:* Aggiunta/Rimozione persona dal gruppo (Op. meno frequente).

Tavola degli accessi (Senza Ridondanza) Per ottenere il totale partecipanti, bisogna accedere a GRUPPO (1 accesso) e poi leggere tutte le righe correlate in PERSONA (N accessi).

- Costo Lettura: Alto ($1 + N$)
- Costo Scrittura: Basso (1 accesso per inserire la persona)

Tavola degli accessi (Con Ridondanza) Il valore è memorizzato direttamente in GRUPPO.

- Costo Lettura: Basso (1 accesso diretto all'entità Gruppo).
- Costo Scrittura: Doppio (1 accesso per inserire la persona + 1 accesso per aggiornare il contatore in Gruppo).

Conclusione: Dato che l'operazione di lettura dei dettagli del gruppo è molto più frequente rispetto alla modifica della composizione del gruppo stesso, si è deciso di **mantenere la ridondanza** inserendo l'attributo calcolato nell'entità GRUPPO per ottimizzare le prestazioni in lettura.

5.2 Eliminazione delle generalizzazioni

Le gerarchie presenti nello schema concettuale sono state eliminate accorpando le entità figlie nell'entità genitore :

- **Parco:** Le specializzazioni (Naturale, Riserva, ecc.) sono state accorpate in **PARCO** aggiungendo l'attributo discriminante **tipo**.
- **Persona:** Le entità **BAMBINO** e **SENIOR** sono state accorpate in **PERSONA** aggiungendo l'attributo **tipo** (Utente/Guida/Visitatore).
- **Tragitto:** Le specializzazioni (Bici, Piedi, Cavallo) sono state accorpate in **TRAGITTI** tramite l'attributo **tipo**. Questa scelta semplifica la gestione delle query e riduce il numero di join necessari.

5.3 Partizionamento/Accorpamento

Non sono stati effettuati partizionamenti verticali o orizzontali in quanto le entità risultanti non presentano un numero di attributi tale da giustificare una separazione per motivi di efficienza.

5.4 Eliminazione degli attributi composti

- **Indirizzo (Centro Visita):** L'attributo composto è stato trasformato in una stringa unica concatenata.
- **Indirizzo (Struttura Ricettiva):** In questo caso specifico, i sotto-attributi (**Via**, **numCivico**, **CAP**) sono stati mantenuti separati poiché costituiscono parte della Chiave Primaria.

5.5 Scelta degli identificatori principali

Per la maggior parte delle entità sono stati mantenuti gli identificatori naturali (es. **nomeParco**, **username**, **COD_fiscale**) in quanto univoci e stabili nel dominio applicativo. Per entità come **FEEDBACK** o **GUIDA** (Tesserino) si è optato per codici o ID univoci per semplificare le referenze esterne .

6. Codice SQL

6.1 Creazione DB e tuple

SQL

```
CREATE TABLE PARCO (  
    nomeParco VARCHAR(255) NOT NULL,  
    descrizione TEXT NOT NULL,  
    superficie INTEGER NOT NULL,  
    regione VARCHAR(255) NOT NULL,  
    accessibile BOOLEAN NOT NULL,  
    tipo VARCHAR(255) NOT NULL,  
    PRIMARY KEY (nomeParco)  
);
```

```
CREATE TABLE ENTE (  
    nomeEnte VARCHAR(255) NOT NULL,  
    CETS BOOLEAN NOT NULL,  
    PRIMARY KEY (nomeEnte)  
);
```

```
CREATE TABLE STRUTTURA_RICETTIVA (  
    Via VARCHAR(255) NOT NULL,  
    numCivico INTEGER NOT NULL,  
    CAP INTEGER NOT NULL,  
    nome VARCHAR(255) NOT NULL,  
    contatti VARCHAR(255) NOT NULL,  
    servizi TEXT NOT NULL,  
    impiego_ecologico BOOLEAN,
```

```
    trattamenti VARCHAR(255) NOT NULL,  
    parcheggio BOOLEAN NOT NULL,  
    CETS BOOLEAN,  
    PRIMARY KEY (Via, numCivico, CAP)  
);
```

```
CREATE TABLE TRAGITTI (  
    nomeTragitto VARCHAR(255) NOT NULL,  
    durata TIME NOT NULL,  
    punto_partenza VARCHAR(255) NOT NULL,  
    difficolta INTEGER NOT NULL,  
    descrizione TEXT NOT NULL,  
    tipo VARCHAR(255) NOT NULL,  
    PRIMARY KEY (nomeTragitto)  
);
```

```
CREATE TABLE TOUR (  
    nomeTour VARCHAR(255) NOT NULL,  
    data DATE NOT NULL,  
    ora TIME NOT NULL,  
    stato VARCHAR(255) NOT NULL,  
    maxPartecipanti INTEGER NOT NULL,  
    PRIMARY KEY (nomeTour)  
);
```

```
CREATE TABLE GRUPPO (  
    nomeGruppo VARCHAR(255) NOT NULL,
```

```
    tipo VARCHAR(255) NOT NULL,  
    classe VARCHAR(255),  
    PRIMARY KEY (nomeGruppo)  
);
```

```
CREATE TABLE GUIDA (  
    numTesserino INTEGER NOT NULL,  
    voto INTEGER NOT NULL,  
    licenza VARCHAR(255) NOT NULL,  
    campo VARCHAR(255) NOT NULL,  
    PRIMARY KEY (numTesserino)  
);
```

```
CREATE TABLE CALENDARIO (  
    data DATE NOT NULL,  
    oraInizio TIME NOT NULL,  
    oraFine TIME NOT NULL,  
    note TEXT NOT NULL,  
    PRIMARY KEY (data, oraInizio, oraFine)  
);
```

```
CREATE TABLE CENTRO_VISITA (  
    nomeCentro VARCHAR(255) NOT NULL,  
    indirizzo VARCHAR(255) NOT NULL,  
    orarioApertura TIME NOT NULL,  
    servizi TEXT NOT NULL,  
    note TEXT NOT NULL,
```

```

        orarioChiusura TIME NOT NULL,
        telefono VARCHAR(255) NOT NULL,
        email VARCHAR(255) NOT NULL,
        PRIMARY KEY (nomeCentro)
    );

CREATE TABLE PERSONA (
    COD_fiscale VARCHAR(16) NOT NULL,
    cognome VARCHAR(255) NOT NULL,
    nome VARCHAR(255) NOT NULL,
    dataNascita DATE NOT NULL,
    tipo VARCHAR(255) NOT NULL,
    GRUPPO_nomeGruppo VARCHAR(255),
    PRIMARY KEY (COD_fiscale),
    FOREIGN KEY (GRUPPO_nomeGruppo) REFERENCES GRUPPO
(nomeGruppo)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

CREATE TABLE UTENTE (
    username VARCHAR(255) NOT NULL,
    password VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    telefono VARCHAR(255) NOT NULL,
    PERSONA_COD_fiscale VARCHAR(16) NOT NULL,
    PRIMARY KEY (username),

```

```
        FOREIGN KEY (PERSONA_COD_fiscale) REFERENCES PERSONA
(COD_fiscale)

        ON DELETE NO ACTION

        ON UPDATE NO ACTION

);
```

```
CREATE TABLE NEWS (

    data DATE NOT NULL,

    titolo VARCHAR(255) NOT NULL,

    autore VARCHAR(255) NOT NULL,

    testo TEXT NOT NULL,

    foto VARCHAR(255) NOT NULL,

    PARCO_nomeParco VARCHAR(255) NOT NULL,

    PRIMARY KEY (data, titolo),

    FOREIGN KEY (PARCO_nomeParco) REFERENCES PARCO (nomeParco)

    ON DELETE NO ACTION

    ON UPDATE NO ACTION

);
```

```
CREATE TABLE FEEDBACK (

    TRAGITTI_nomeTragitto VARCHAR(255) NOT NULL,

    data DATE NOT NULL,

    ora TIME NOT NULL,

    valutazione INTEGER NOT NULL,

    commento TEXT NOT NULL,

    tipo VARCHAR(255) NOT NULL,

    UTENTE_username VARCHAR(255),
```

```

    anonimo BOOLEAN NOT NULL,

    PRIMARY KEY (TRAGITTI_nomeTragitto, data, ora),

    FOREIGN KEY (TRAGITTI_nomeTragitto) REFERENCES TRAGITTI
(nomeTragitto)

    ON DELETE NO ACTION

    ON UPDATE NO ACTION,

    FOREIGN KEY (UTENTE_username) REFERENCES UTENTE (username)

    ON DELETE NO ACTION

    ON UPDATE NO ACTION,

    CONSTRAINT chk_feedback_anonimo CHECK (

        (anonimo = true AND UTENTE_username IS NULL) OR

        (anonimo = false AND UTENTE_username IS NOT NULL)

    )

);

```

```

CREATE TABLE GESTIONE (

    PARCO_nomeParco VARCHAR(255) NOT NULL,

    ENTE_nomeEnte VARCHAR(255) NOT NULL,

    PRIMARY KEY (PARCO_nomeParco, ENTE_nomeEnte),

    FOREIGN KEY (PARCO_nomeParco) REFERENCES PARCO (nomeParco)

    ON DELETE NO ACTION

    ON UPDATE NO ACTION,

    FOREIGN KEY (ENTE_nomeEnte) REFERENCES ENTE (nomeEnte)

    ON DELETE NO ACTION

    ON UPDATE NO ACTION

);

```

```

CREATE TABLE RISERVARE (
    STRUTTURA_RICETTIVA_Via VARCHAR(255) NOT NULL,
    STRUTTURA_RICETTIVA_numCivico INTEGER NOT NULL,
    STRUTTURA_RICETTIVA_CAP INTEGER NOT NULL,
    PERSONA_COD_fiscale VARCHAR(16) NOT NULL,
    dataPrenotazione DATE NOT NULL,
    numStanze INTEGER NOT NULL,
    stato VARCHAR(255) NOT NULL,

    PRIMARY KEY (STRUTTURA_RICETTIVA_Via,
    STRUTTURA_RICETTIVA_numCivico, STRUTTURA_RICETTIVA_CAP,
    PERSONA_COD_fiscale),

    FOREIGN KEY (STRUTTURA_RICETTIVA_Via,
    STRUTTURA_RICETTIVA_numCivico, STRUTTURA_RICETTIVA_CAP)
    REFERENCES STRUTTURA_RICETTIVA (Via, numCivico, CAP)

    ON DELETE NO ACTION

    ON UPDATE NO ACTION,

    FOREIGN KEY (PERSONA_COD_fiscale) REFERENCES PERSONA
    (COD_fiscale)

    ON DELETE NO ACTION

    ON UPDATE NO ACTION
);

CREATE TABLE INCLUDE (
    TRAGITTI_nomeTragitto VARCHAR(255) NOT NULL,
    TOUR_nomeTour VARCHAR(255) NOT NULL,

    PRIMARY KEY (TRAGITTI_nomeTragitto, TOUR_nomeTour),

    FOREIGN KEY (TRAGITTI_nomeTragitto) REFERENCES TRAGITTI
    (nomeTragitto)

    ON DELETE NO ACTION

```

```

        ON UPDATE NO ACTION,
        FOREIGN KEY (TOUR_nomeTour) REFERENCES TOUR (nomeTour)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION
    );

CREATE TABLE VISITA (
    PARCO_nomeParco VARCHAR(255) NOT NULL,
    PERSONA_COD_fiscale VARCHAR(16) NOT NULL,
    data DATE NOT NULL,
    oraEntrata TIME NOT NULL,
    oraUscita TIME NOT NULL,
    PRIMARY KEY (PARCO_nomeParco, PERSONA_COD_fiscale),
    FOREIGN KEY (PARCO_nomeParco) REFERENCES PARCO (nomeParco)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
    FOREIGN KEY (PERSONA_COD_fiscale) REFERENCES PERSONA
(COD_fiscale)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

```

```

CREATE TABLE PRENOTA (
    TOUR_nomeTour VARCHAR(255) NOT NULL,
    UTENTE_username VARCHAR(255) NOT NULL,
    data DATE NOT NULL,
    orario TIME NOT NULL,

```

```

giorniDisponibili INTEGER NOT NULL,
disponibile BOOLEAN NOT NULL,
stato VARCHAR(255) NOT NULL,
PRIMARY KEY (TOUR_nomeTour, UTENTE_username),
FOREIGN KEY (TOUR_nomeTour) REFERENCES TOUR (nomeTour)
ON DELETE NO ACTION
ON UPDATE NO ACTION,
FOREIGN KEY (UTENTE_username) REFERENCES UTENTE (username)
ON DELETE NO ACTION
ON UPDATE NO ACTION
);

```

```

CREATE TABLE ASSEGNATA_A (
    TOUR_nomeTour VARCHAR(255) NOT NULL,
    GUIDA_numTesserino INTEGER NOT NULL,
    PRIMARY KEY (TOUR_nomeTour, GUIDA_numTesserino),
    FOREIGN KEY (TOUR_nomeTour) REFERENCES TOUR (nomeTour)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
    FOREIGN KEY (GUIDA_numTesserino) REFERENCES GUIDA
(numTesserino)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

```

```

CREATE TABLE SVOLGE (
    TRAGITTI_nomeTragitto VARCHAR(255) NOT NULL,

```

```

        GUIDA_numTesserino INTEGER NOT NULL,

        PRIMARY KEY (TRAGITTI_nomeTragitto, GUIDA_numTesserino),

        FOREIGN KEY (TRAGITTI_nomeTragitto) REFERENCES TRAGITTI
(nomeTragitto)

        ON DELETE NO ACTION

        ON UPDATE NO ACTION,

        FOREIGN KEY (GUIDA_numTesserino) REFERENCES GUIDA
(numTesserino)

        ON DELETE NO ACTION

        ON UPDATE NO ACTION

    );

```

```

CREATE TABLE DISPONIBILE (

    GUIDA_numTesserino INTEGER NOT NULL,

    CALENDARIO_data DATE NOT NULL,

    CALENDARIO_oraInizio TIME NOT NULL,

    CALENDARIO_oraFine TIME NOT NULL,

    PRIMARY KEY (GUIDA_numTesserino, CALENDARIO_data,
CALENDARIO_oraInizio, CALENDARIO_oraFine),

    FOREIGN KEY (GUIDA_numTesserino) REFERENCES GUIDA
(numTesserino)

    ON DELETE NO ACTION

    ON UPDATE NO ACTION,

    FOREIGN KEY (CALENDARIO_data, CALENDARIO_oraInizio,
CALENDARIO_oraFine) REFERENCES CALENDARIO (data, oraInizio,
oraFine)

    ON DELETE NO ACTION

    ON UPDATE NO ACTION

);

```

```

CREATE TABLE SI_RIVOLGE_A (
    PERSONA_COD_fiscale VARCHAR(16) NOT NULL,
    CENTRO_VISITA_nomeCentro VARCHAR(255) NOT NULL,
    PRIMARY KEY (PERSONA_COD_fiscale,
CENTRO_VISITA_nomeCentro),
    FOREIGN KEY (PERSONA_COD_fiscale) REFERENCES PERSONA
(COD_fiscale)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
    FOREIGN KEY (CENTRO_VISITA_nomeCentro) REFERENCES
CENTRO_VISITA (nomeCentro)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

CREATE TABLE SI_RIFERISCE_A (
    PARCO_nomeParco VARCHAR(255) NOT NULL,
    CENTRO_VISITA_nomeCentro VARCHAR(255) NOT NULL,
    PRIMARY KEY (PARCO_nomeParco, CENTRO_VISITA_nomeCentro),
    FOREIGN KEY (PARCO_nomeParco) REFERENCES PARCO (nomeParco)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
    FOREIGN KEY (CENTRO_VISITA_nomeCentro) REFERENCES
CENTRO_VISITA (nomeCentro)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);

```

```
CREATE TABLE RELATIVO_A (  
    GUIDA_numTesserino INTEGER NOT NULL,  
    FEEDBACK_TRAGITTI_nomeTragitto VARCHAR(255) NOT NULL,  
    FEEDBACK_data DATE NOT NULL,  
    FEEDBACK_ora TIME NOT NULL,  
    PRIMARY KEY (GUIDA_numTesserino,  
    FEEDBACK_TRAGITTI_nomeTragitto, FEEDBACK_data, FEEDBACK_ora),  
    FOREIGN KEY (GUIDA_numTesserino) REFERENCES GUIDA  
    (numTesserino)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION,  
    FOREIGN KEY (FEEDBACK_TRAGITTI_nomeTragitto,  
    FEEDBACK_data, FEEDBACK_ora) REFERENCES FEEDBACK  
    (TRAGITTI_nomeTragitto, data, ora)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION  
);
```

6.2 Inserimento in tabella

SQL

-- PERSONA

insert into Persona values

('RSSMRA80A01H501Z', 'Rossi', 'Mario', '1980-01-01',
'Visitatore', NULL),

('BNCLRA95C10H501X', 'Bianchi', 'Laura', '1995-03-10',
'Guida', NULL),

('VRDGPP00D15H501A', 'Verdi', 'Giuseppe', '2000-04-15',
'Utente', 'Classe3A');

-- ENTE

insert into Ente values

('WWF', true),

('Legambiente', false);

-- PARCO

insert into Parco values

('Gran Paradiso', 'Parco nazionale alpino', 70300, 'Piemonte',
true, 'Montano'),

('Cinque Terre', 'Area costiera ligure', 3800, 'Liguria',
true, 'Marino');

-- GRUPPO

insert into Gruppo values

('Classe3A', 'Scolastico', 25, '3A'),

('TrekkingClub', 'Amatoriale', 12, null);

-- UTENTE

```

insert into Utente values

('marior', 'password1', 'mario.rossi@email.it', '333123456',
'RSSMRA80A01H501Z'),

('peppev', 'passverde', 'g.verdi@email.it', '339111222',
'VRDGPP00D15H501A');

-- GUIDA

insert into Guida values

(101, 5, 'LIC123', 'Naturalistica'),

(102, 4, 'LIC456', 'Storica');

-- TRAGITTI

insert into Tragitti values

('SentieroLago', '02:30:00', 'Base rifugio', 4, 'Percorso
intorno al lago alpino', 'Trekking'),

('BorgoAntico', '01:00:00', 'Centro visite', 2, 'Passeggiata
nel borgo medievale', 'Culturale');

-- FEEDBACK

insert into Feedback values

('SentieroLago', '2025-10-01', '09:30:00', 5, 'Bellissimo
percorso!', 'Positivo', 'marior', false),

('BorgoAntico', '2025-10-02', '11:00:00', 3, 'Troppo breve ma
interessante', 'Neutro', null, true),

('SentieroLago', '2025-11-15', '10:00:00', 5, 'Tutto
perfetto!', 'Positivo', 'peppev', false),

('BorgoAntico', '2025-11-16', '11:00:00', 4, 'Molto carino,
consigliato.', 'Positivo', NULL, true);

-- Riservare una struttura esistente (CORRETTO)

```

```

insert into Riservare values

('ViaMontagna', 12, 10100, 'BNCLRA95C10H501X', '2025-12-05',
1, 'Confermata');


-- CALENDARIO

insert into Calendario values

('2025-10-20', '09:00:00', '12:00:00', 'Tour mattutino'),
('2025-10-21', '14:00:00', '17:00:00', 'Visita pomeridiana');


-- TOUR

insert into Tour values

('TourLago', '2025-10-20', '09:00:00', 'Confermato', 15),
('TourBorgo', '2025-10-21', '14:00:00', 'Aperto', 20);


-- ASSEGNATA_A

insert into Assegnata_a values

('TourLago', 101),
('TourBorgo', 102);


-- INCLUDE

insert into Include values

('SentieroLago', 'TourLago'),
('BorgoAntico', 'TourBorgo');


-- SVOLGE

insert into Svolge values

('SentieroLago', 101),

```

```

('BorgoAntico', 102);

-- RELATIVO_A
insert into Relativo_a values
(101, 'SentieroLago', '2025-10-01', '09:30:00'),
(102, 'BorgoAntico', '2025-10-02', '11:00:00');

-- STRUTTURA_RICETTIVA
insert into Struttura_ricettiva values
('ViaMontagna', 12, 10100, 'HotelAlpino', '0123456789',
'WiFi', true, 'Mezza pensione', true, true),
('ViaLitorale', 5, 19010, 'B&B MareBlu', '019654321',
'Colazione', false, 'Solo pernottamento', false, false);

-- RISERVARE
insert into Riservare values
('ViaMontagna', 12, 10100, 'RSSMRA80A01H501Z', '2025-10-10',
1, 'Confermata'),
('ViaLitorale', 5, 19010, 'VRDGPP00D15H501A', '2025-10-11', 2,
'In attesa');

-- CENTRO_VISITA
insert into Centro_visita values
('CentroGranParadiso', 'PiazzaAlpi', '08:00:00', 'Info,
Mostre', 'Aperto tutto l'anno', '18:00:00', '0123456789',
'info@gp.it'),
('CentroCinqueTerre', 'ViaMarina', '09:00:00', 'Info, Guida',
'Chiuso martedì', '19:00:00', '019112233', 'info@5terre.it');

```

```
-- SI_RIFERISCE_A
```

```
insert into Si_riferisce_a values
```

```
('Gran Paradiso', 'CentroGranParadiso'),
```

```
('Cinque Terre', 'CentroCinqueTerre');
```

```
-- SI_RIVOLGE_A
```

```
insert into Si_rivolge_a values
```

```
('RSSMRA80A01H501Z', 'CentroGranParadiso'),
```

```
('VRDGPP00D15H501A', 'CentroCinqueTerre');
```

```
-- GESTIONE
```

```
insert into Gestione values
```

```
('Gran Paradiso', 'WWF'),
```

```
('Cinque Terre', 'Legambiente');
```

```
-- PRENOTA
```

```
insert into Prenota values
```

```
('TourLago', 'marior', '2025-10-01', '09:00:00', 1, true,  
'Confermato'),
```

```
('TourBorgo', 'peppev', '2025-10-05', '14:00:00', 1, true, 'In  
attesa');
```

6.3 Query Test

SQL

```
-- =====  
--   TEST DATABASE QUERY SUITE  
-- =====  
  
-- ① Query base  
SELECT * FROM Persona;  
  
SELECT u.username, u.email, p.nome, p.cognome, p.tipo  
FROM Utente u  
JOIN Persona p ON p.COD_fiscale = u.PERSONA_COD_fiscale;  
  
SELECT numTesserino, licenza, campo, voto FROM Guida;  
  
-- ② Parchi e gestione  
SELECT pa.nomeParco, e.nomeEnte, e.CETS  
FROM Parco pa  
JOIN Gestione g ON pa.nomeParco = g.PARCO_nomeParco  
JOIN Ente e ON g.ENTE_nomeEnte = e.nomeEnte;  
  
SELECT c.nomeCentro, p.nomeParco, p.regione  
FROM Centro_visita c  
JOIN Si_riferisce_a s ON c.nomeCentro = s.CENTRO_VISITA_nomeCentro  
JOIN Parco p ON s.PARCO_nomeParco = p.nomeParco;  
  
-- ③ Tragitti, tour e guide  
SELECT t.nomeTour, tr.nomeTragitto, tr.durata, tr.difficolta  
FROM Tour t  
JOIN Include i ON t.nomeTour = i.TOUR_nomeTour  
JOIN Tragitti tr ON i.TRAGITTI_nomeTragitto = tr.nomeTragitto;  
  
SELECT g.numTesserino, g.campo, tr.nomeTragitto, tr.tipo  
FROM Guida g  
JOIN Svolge s ON g.numTesserino = s.GUIDA_numTesserino  
JOIN Tragitti tr ON s.TRAGITTI_nomeTragitto = tr.nomeTragitto;
```

```

SELECT t.nomeTour, t.data, g.numTesserino, g.licenza, g.voto
FROM Tour t
JOIN Assegnata_a a ON t.nomeTour = a.TOUR_nomeTour
JOIN Guida g ON a.GUIDA_numTesserino = g.numTesserino;

```

-- ④ Feedback e utenti

```

SELECT f.data, f.ora, f.TRAGITTI_nomeTragitto, f.valutazione,
f.commento, f.UTENTE_username
FROM Feedback f
WHERE f.anonimo = false;

```

```

SELECT TRAGITTI_nomeTragitto, AVG(valutazione) AS media_valutazioni
FROM Feedback
GROUP BY TRAGITTI_nomeTragitto;

```

```

SELECT DISTINCT u.username, p.nome, p.cognome
FROM Feedback f
JOIN Utente u ON f.UTENTE_username = u.username
JOIN Persona p ON u.PERSONA_COD_fiscale = p.COD_fiscale;

```

-- ⑤ Strutture e prenotazioni

```

SELECT r.PERSONA_COD_fiscale, p.nome, p.cognome, s.nome AS
struttura, r.dataPrenotazione, r.stato
FROM RISERVARE r
JOIN Persona p ON r.PERSONA_COD_fiscale = p.COD_fiscale
JOIN Struttura_ricettiva s ON (r.STRUTTURA_RICETTIVA_Via,
r.STRUTTURA_RICETTIVA_CAP, r.STRUTTURA_RICETTIVA_numCivico) =
(s.Via, s.CAP, s.numCivico);

```

```

SELECT s.nome, COUNT(*) AS numero_prenotazioni
FROM Struttura_ricettiva s
JOIN Riservare r ON (s.Via, s.CAP, s.numCivico) =
(r.STRUTTURA_RICETTIVA_Via, r.STRUTTURA_RICETTIVA_CAP,
r.STRUTTURA_RICETTIVA_numCivico)
GROUP BY s.nome;

```

-- ⑥ Gruppi e appartenenze

```
SELECT g.nomeGruppo, p.nome, p.cognome
FROM Persona p
JOIN Gruppo g ON p.GRUPPO_nomeGruppo = g.nomeGruppo
ORDER BY g.nomeGruppo;
```

-- 7 Query complesse

```
SELECT u.username, t.nomeTour, t.data, g.numTesserino, g.campo
FROM PRENOTA pr
JOIN Utente u ON pr.UTENTE_username = u.username
JOIN Tour t ON pr.TOUR_nomeTour = t.nomeTour
JOIN Assegnata_a a ON t.nomeTour = a.TOUR_nomeTour
JOIN Guida g ON a.GUIDA_numTesserino = g.numTesserino
ORDER BY u.username;
```

```
SELECT TRAGITTI_nomeTragitto, AVG(valutazione) AS media
FROM Feedback
GROUP BY TRAGITTI_nomeTragitto
HAVING AVG(valutazione) > 4;
```

```
SELECT p.nomeParco, e.nomeEnte
FROM Gestione g
JOIN Parco p ON g.PARCO_nomeParco = p.nomeParco
JOIN Ente e ON g.ENTE_nomeEnte = e.nomeEnte
WHERE e.CETS = true;
```

6.4 Test dei vincoli in SQL

SQL

-- VERIFICA DI COERENZA

SELECT * FROM Feedback

WHERE (anonimo = true AND UTENTE_username IS NOT NULL)
OR (anonimo = false AND UTENTE_username IS NULL);

SELECT *

FROM Riservare r

WHERE (r.STRUTTURA_RICETTIVA_Via, r.STRUTTURA_RICETTIVA_CAP,
r.STRUTTURA_RICETTIVA_numCivico) NOT IN (
SELECT Via, CAP, numCivico FROM Struttura_ricettiva);

-- VERIFICA DI VIOLAZIONE ATTIVA

DELETE FROM PARCO WHERE nomeParco = 'Gran Paradiso';

INSERT INTO Feedback (TRAGITTI_nomeTragitto, data, ora,
valutazione, commento, tipo, UTENTE_username, anonimo)

VALUES ('SentieroLago', '2025-12-01', '10:00:00', 1, 'Fake',
'Negativo', 'utente_inesistente_99', false);